



Advanced Browser APIs — ব্রাউজার ও ডিভাইস সক্ষমতা ব্যবহারের মূল উদ্দেশ্য

JavaScript-এর কোর ল্যঙ্গুয়েজের সীমানা ছাড়িয়ে ব্রাউজার রানটাইম ডেভেলপারদের যেসব শক্তিশালী প্ল্যাটফর্ম এপিআই প্রদান করে, আধুনিক ওয়েব অ্যাপ্লিকেশনের প্রাণশক্তি হলো সেগুলো। এই চ্যাপ্টারে আমরা **Geolocation**, **IntersectionObserver**, **MutationObserver**, **ResizeObserver**, **File API & Blob**, **Web Workers**, **BroadcastChannel**, **IndexedDB** এবং **WebSockets** গভীরভাবে শিখব।

✦ 1. Geolocation API

✦ 2. URL & History

✦ 3. Clipboard API

✦ 4. 3 Observers Trio

✦ 5. File API & Blob

✦ 6. Web Workers

✦ 7. BroadcastChannel

✦ 8. IndexedDB

✦ 9. WebSockets

✦ 10. Wake Lock & Share

Part 01 — Browser Web APIs Architecture & Geolocation Integration (32.1 – 32.2)

32.1 Browser API কী?

Browser API হলো browser-এর দেওয়া JavaScript interface, যার মাধ্যমে JavaScript browser-এর বিভিন্ন capability ব্যবহার করতে পারে। যেমন:

JAVASCRIPT SOURCE

JAVASCRIPT

```
navigator.geolocation;
```

এর মাধ্যমে location access করা যায়।

JAVASCRIPT SOURCE

JAVASCRIPT

```
navigator.clipboard;
```

এর মাধ্যমে clipboard ব্যবহার করা যায়।

JAVASCRIPT SOURCE

JAVASCRIPT

```
window.history;
```

এর মাধ্যমে browser history পরিবর্তন করা যায়।

◆ সহজভাবে

JavaScript নিজে জানে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
let x = 10;  
console.log(x);
```

কিন্তু browser JavaScript-কে extra ক্ষমতা দেয়:

JAVASCRIPT SOURCE

JAVASCRIPT

```
navigator.geolocation;  
navigator.clipboard;  
window.history;  
window.localStorage;
```

32.2 Geolocation API

Website থেকে user's geographic location নেওয়ার জন্য Geolocation API ব্যবহার করা হয়।

♦ Basic Syntax

JAVASCRIPT SOURCE

JAVASCRIPT

```
navigator.geolocation.getCurrentPosition(success, error);
```

♦ Example

JAVASCRIPT SOURCE

JAVASCRIPT

```
navigator.geolocation.getCurrentPosition(
  (position) => {
    console.log(position.coords.latitude);
    console.log(position.coords.longitude);
  },
  (error) => {
    console.log(error.message);
  },
);
```

♦ Output

🌐 BROWSER API ARCHITECTURE & DATA FLOW

```
23.8103
90.4125
```

এগুলো latitude এবং longitude।

♦ Real-life Example

Food delivery website:

🌐 GEOLOCATION HARDWARE INTEGRATION FLOW

```
Current Location
  ↓
GPS
  ↓
Latitude + Longitude
  ↓
Nearby Restaurants
```

🌐 Position-এর গুরুত্বপূর্ণ properties

JAVASCRIPT SOURCE

JAVASCRIPT

```
position.coords.latitude;
position.coords.longitude;
position.coords.accuracy;
position.coords.altitude;
position.coords.speed;
```

সবসময় সব property পাওয়া যাবে এমন নয়।

♦ Better Example

JAVASCRIPT SOURCE

JAVASCRIPT

```
function getLocation() {
  navigator.geolocation.getCurrentPosition(
    (position) => {
      const { latitude, longitude } = position.coords;

      console.log(`Latitude: ${latitude}`);
      console.log(`Longitude: ${longitude}`);
    },
    (error) => {
      console.error("Location error:", error.message);
    },
  );
}

getLocation();
```

🌐 watchPosition()

User-এর location continuously track করার জন্য:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const watchId = navigator.geolocation.watchPosition((position) => {  
  console.log(position.coords.latitude);  
});
```

Stop করতে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
navigator.geolocation.clearWatch(watchId);
```

◆ Real-life

Ride-sharing app:

🌐 BROWSER API ARCHITECTURE & DATA FLOW

```
Driver moves  
↓  
GPS updates  
↓  
watchPosition()  
↓  
Map updates
```

⚠️ Location permission এবং privacy অত্যন্ত গুরুত্বপূর্ণ!

Part 02 — URL Processing, Query Strings & History Routing (32.3 – 32.6)

32.3 URL API

JavaScript-এর URL object দিয়ে URL parse এবং manipulate করা যায়।

JAVASCRIPT SOURCE

JAVASCRIPT

```
const url = new URL("https://example.com/products?id=101&category=phone");  
  
console.log(url.hostname);  
console.log(url.pathname);  
console.log(url.search);
```

◆ Output

🌐 BROWSER API ARCHITECTURE & DATA FLOW

```
example.com  
/products  
?id=101&category=phone
```

🌐 URL-এর গুরুত্বপূর্ণ properties

JAVASCRIPT SOURCE

JAVASCRIPT

```
url.href;  
url.protocol;  
url.host;  
url.hostname;  
url.port;  
url.pathname;  
url.search;  
url.hash;
```

Example:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const url = new URL("https://example.com:8080/products?id=10#details");  
  
console.log(url.protocol); // https:  
console.log(url.hostname); // example.com  
console.log(url.port); // 8080  
console.log(url.pathname); // /products  
console.log(url.search); // ?id=10  
console.log(url.hash); // #details
```

32.4 URLSearchParams

Query parameters handle করার জন্য URLSearchParams খুব usefull

JAVASCRIPT SOURCE

JAVASCRIPT

```
const params = new URLSearchParams("?name=Ripon&age=24");

console.log(params.get("name"));
console.log(params.get("age"));
```

Output:



BROWSER API ARCHITECTURE & DATA FLOW

Ripon
24

Add

JAVASCRIPT SOURCE

JAVASCRIPT

```
params.set("city", "Dhaka");
```

Delete

JAVASCRIPT SOURCE

JAVASCRIPT

```
params.delete("age");
```

Check

JAVASCRIPT SOURCE

JAVASCRIPT

```
params.has("name");
```

Convert to string

JAVASCRIPT SOURCE

JAVASCRIPT

```
console.log(params.toString());
```

Real-life API example

JAVASCRIPT SOURCE

JAVASCRIPT

```
const params = new URLSearchParams({
  search: "laptop",
  page: 2,
  limit: 20,
});

const url = `/products?${params}`;

console.log(url);
```

Output:



BROWSER API ARCHITECTURE & DATA FLOW

/products?search=laptop&page=2&limit=20

🔥 React/Next.js/Node.js API development-এ এটি খুব common!

32.5 History API

Browser-এর history manipulate করার জন্য:

JAVASCRIPT SOURCE

JAVASCRIPT

```
window.history;
```

ব্যবহার করা হয়। Important methods:

JAVASCRIPT SOURCE

JAVASCRIPT

```
history.back();
history.forward();
history.go();
history.pushState();
history.replaceState();
```

pushState()

নতুন history entry তৈরি করে।

JAVASCRIPT SOURCE

JAVASCRIPT

```
history.pushState({ page: 2 }, "", "/products?page=2");
```

Browser URL পরিবর্তন হবে:

 BROWSER API ARCHITECTURE & DATA FLOW

```
/products?page=2
```

কিন্তু page reload হবে না।

replaceState()

বর্তমান history entry replace করে।

JAVASCRIPT SOURCE

JAVASCRIPT

```
history.replaceState({ page: 2 }, "", "/products?page=2");
```

32.6 popstate Event

Back/forward navigation detect করতে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
window.addEventListener("popstate", (event) => {
  console.log(event.state);
});
```

♦ SPA-তে ব্যবহার

React Router-এর মতো routing system-এর concept বুঝতে History API জানা গুরুত্বপূর্ণ।

32.7 Clipboard API

Copy/paste functionality তৈরি করতে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
navigator.clipboard;
```

ব্যবহার করা হয়।

♦ Copy

JAVASCRIPT SOURCE

JAVASCRIPT

```
navigator.clipboard.writeText("Hello World");
```

♦ Read

JAVASCRIPT SOURCE

JAVASCRIPT

```
navigator.clipboard.readText().then((text) => {
  console.log(text);
});
```

♦ Better example

JAVASCRIPT SOURCE

JAVASCRIPT

```
async function copyText() {
  try {
    await navigator.clipboard.writeText("Hello Ripon");
    console.log("Copied!");
  } catch (error) {
    console.error("Copy failed:", error);
  }
}

copyText();
```

♦ Real-life

🌐 BROWSER API ARCHITECTURE & DATA FLOW

```
API Key
  ↓
[ Copy ]
  ↓
Clipboard API
```

GitHub, password managers, developer tools ইত্যাদিতে এই concept দেখা যায়।

32.8 Notifications API

Browser notification দেখানোর জন্য:

JAVASCRIPT SOURCE

JAVASCRIPT

```
Notification;
```

◆ Permission

JAVASCRIPT SOURCE

JAVASCRIPT

```
Notification.requestPermission();
```

◆ Notification

JAVASCRIPT SOURCE

JAVASCRIPT

```
new Notification("Hello!");
```

🌐 Complete example

JAVASCRIPT SOURCE

JAVASCRIPT

```
async function showNotification() {  
  const permission = await Notification.requestPermission();  
  
  if (permission === "granted") {  
    new Notification("New Message", {  
      body: "You received a new message.",  
    });  
  }  
}  
  
showNotification();
```

Possible permission:

🌐 BROWSER API ARCHITECTURE & DATA FLOW

```
granted  
denied  
default
```

⚠️ Browser permission এবং user privacy rules গুরুত্বপূর্ণ।

32.9 IntersectionObserver

এটি detect করতে পারে কোনো element viewport-এর মধ্যে এসেছে কিনা।

♦ Basic Example

JAVASCRIPT SOURCE

JAVASCRIPT

```
const observer = new IntersectionObserver((entries) => {
  entries.forEach((entry) => {
    if (entry.isIntersecting) {
      console.log("Element visible!");
    }
  });
});

const box = document.querySelector(".box");

observer.observe(box);
```

🌐 Real-life use

♦ Lazy loading

🌐 VIEWPORT INTERSECTION OBSERVER LIFECYCLE

```
Page
↓
Image এখনো screen-এর বাইরে
↓
User scroll করে
↓
Image viewport-এ আসে
↓
IntersectionObserver
↓
Image load
```

🌐 Lazy image example

HTML DOM MARKUP

HTML

```

```

JAVASCRIPT SOURCE

JAVASCRIPT

```
const observer = new IntersectionObserver((entries) => {
  entries.forEach((entry) => {
    if (entry.isIntersecting) {
      const img = entry.target;

      img.src = img.dataset.src;

      observer.unobserve(img);
    }
  });
});

document.querySelectorAll(".lazy").forEach((img) => observer.observe(img));
```

🔥 Performance optimization-এ খুব important!

32.10 MutationObserver

DOM-এর মধ্যে কোনো পরিবর্তন হলে detect করতে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
MutationObserver;
```

ব্যবহার করা হয়।

♦ Example

JAVASCRIPT SOURCE

JAVASCRIPT

```
const observer = new MutationObserver((mutations) => {  
  console.log("DOM changed!");  
});  
  
const target = document.querySelector("#app");  
  
observer.observe(target, {  
  childList: true,  
  subtree: true,  
});
```

♦ Example DOM change

JAVASCRIPT SOURCE

JAVASCRIPT

```
const div = document.querySelector("#app");  
  
div.innerHTML += "<p>Hello</p>";
```

Observer detect করতে পারবে।

♦ Real-life

Dynamic application:

🌐 BROWSER API ARCHITECTURE & DATA FLOW

```
DOM changes  
↓  
MutationObserver  
↓  
Detect changes  
↓  
Run logic
```

32.11 ResizeObserver

কোনো element-এর size পরিবর্তন detect করতে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const observer = new ResizeObserver((entries) => {  
  for (const entry of entries) {  
    console.log(entry.contentRect.width);  
    console.log(entry.contentRect.height);  
  }  
});  
  
observer.observe(document.querySelector(".box"));
```

♦ Real-life

Responsive dashboard:

🌐 BROWSER API ARCHITECTURE & DATA FLOW

```
Sidebar resize  
↓  
Container size changes  
↓  
ResizeObserver  
↓  
Chart resize
```

32.12 IntersectionObserver vs MutationObserver vs ResizeObserver

API	কী observe করে?
IntersectionObserver	Element viewport-এর সাথে intersect করছে কিনা
MutationObserver	DOM change
ResizeObserver	Element size change

🗨️ সহজে মনে রাখুন:

🌐 VIEWPORT INTERSECTION OBSERVER LIFECYCLE

Intersection → Screen visibility
Mutation → DOM changes
Resize → Size changes

Part 05 — File API, FileReader, Binary Blob & Object URLs (32.13 – 32.18)

32.13 File API

Browser-এ user যখন file select করে:

HTML DOM MARKUP

HTML

```
<input type="file" id="fileInput" />
```

JavaScript:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const input = document.querySelector("#fileInput");

input.addEventListener("change", () => {
  const file = input.files[0];

  console.log(file.name);
  console.log(file.size);
  console.log(file.type);
});
```

Possible output:

🌐 BROWSER API ARCHITECTURE & DATA FLOW

photo.jpg
245678
image/jpeg

🌐 File object

Important properties:

JAVASCRIPT SOURCE

JAVASCRIPT

```
file.name;
file.size;
file.type;
file.lastModified;
```

32.14 FileReader

File-এর content পড়তে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const reader = new FileReader();
```

◆ Text file

JAVASCRIPT SOURCE

JAVASCRIPT

```
reader.readAsText(file);
```

তারপর:

JAVASCRIPT SOURCE

JAVASCRIPT

```
reader.onload = () => {  
  console.log(reader.result);  
};
```

Complete:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const input = document.querySelector("#fileInput");  
  
input.addEventListener("change", () => {  
  const file = input.files[0];  
  
  const reader = new FileReader();  
  
  reader.onload = () => {  
    console.log(reader.result);  
  };  
  
  reader.readAsText(file);  
});
```

32.15 Reading Image

Image preview তৈরি করতে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const reader = new FileReader();  
  
reader.onload = () => {  
  img.src = reader.result;  
};  
  
reader.readAsDataURL(file);
```

◆ Real-life

Profile picture upload:

🌐 FILE API & BLOB PROCESSING PIPELINE

```
Select Image  
↓  
File API  
↓  
FileReader  
↓  
Preview  
↓  
Upload
```

32.16 Blob

Blob হলো binary-like data-এর একটি container।

JAVASCRIPT SOURCE

JAVASCRIPT

```
const blob = new Blob(["Hello World"], { type: "text/plain" });
```

তারপর:

JAVASCRIPT SOURCE

JAVASCRIPT

```
console.log(blob.size);  
console.log(blob.type);
```

♦ Real-life

Blob ব্যবহার হতে পারে:

- image data
- PDF data
- generated text
- downloaded files
- binary data

32.17 Object URL

Blob/File-এর জন্য temporary URL তৈরি করা যায়:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const url = URL.createObjectURL(blob);  
  
console.log(url);
```

এটি এমন URL দিতে পারে:

🌐 BROWSER API ARCHITECTURE & DATA FLOW

blob:https://example.com/ ...

♦ Image preview

JAVASCRIPT SOURCE

JAVASCRIPT

```
const imageURL = URL.createObjectURL(file);  
  
img.src = imageURL;
```

কাজ শেষ হলে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
URL.revokeObjectURL(imageURL);
```

🔥 Object URL revoke করা memory management-এর জন্য important!

32.18 Drag and Drop API

HTML:

HTML DOM MARKUP

HTML

```
<div id="dropZone">Drop File Here</div>
```

JavaScript:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const dropZone = document.querySelector("#dropZone");

dropZone.addEventListener("dragover", (event) => {
  event.preventDefault();
});

dropZone.addEventListener("drop", (event) => {
  event.preventDefault();

  const files = event.dataTransfer.files;

  console.log(files);
});
```

◆ Real-life

🌐 BROWSER API ARCHITECTURE & DATA FLOW

```
Computer
  ↓
Drag file
  ↓
Drop Zone
  ↓
dataTransfer.files
  ↓
Upload
```

32.19 Web Workers

JavaScript-এর heavy কাজ main thread block করতে পারে। Web Worker ব্যবহার করলে আলাদা worker context-এ JavaScript চালানো যায়।

♦ Main file

JAVASCRIPT SOURCE

JAVASCRIPT

```
const worker = new Worker("worker.js");

worker.postMessage(1000000000);

worker.onmessage = (event) => {
  console.log(event.data);
};
```

♦ worker.js

JAVASCRIPT SOURCE

JAVASCRIPT

```
self.onmessage = (event) => {
  const number = event.data;

  let result = 0;

  for (let i = 0; i < number; i++) {
    result += i;
  }

  self.postMessage(result);
};
```

♦ Communication

🌐 WEB WORKER MULTI-THREADING ARCHITECTURE

```
Main Thread
  ↓ postMessage()
Worker
  ↓ postMessage()
Main Thread
```

🌐 কেন Worker?

ধরুন:

JAVASCRIPT SOURCE

JAVASCRIPT

```
for (let i = 0; i < 10_000_000_000; i++) {
  // heavy work
}
```

Main thread-এ করলে UI freeze করতে পারে। Worker ব্যবহার করলে main UI responsive রাখা যায়।

♦ Real-life

- image processing
- large calculations
- data processing
- encryption-related computation
- parsing large datasets

⚠️ Worker-এর DOM access সরাসরি করা যায় না।

32.20 BroadcastChannel

একই origin-এর multiple browser contexts-এর মধ্যে message পাঠাতে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const channel = new BroadcastChannel("app_channel");
```

Message পাঠান:

JAVASCRIPT SOURCE

JAVASCRIPT

```
channel.postMessage({  
  type: "logout",  
});
```

অন্য tab:

JAVASCRIPT SOURCE

JAVASCRIPT

```
channel.onmessage = (event) => {  
  console.log(event.data);  
};
```

♦ Real-life

ধরুন একই website দুইটি tab-এ খোলা:



CROSS-TAB BROADCASTCHANNEL SYNC FLOW

```
Tab 1  
  ↓ logout  
BroadcastChannel  
  ↓  
Tab 2  
  ↓  
User logged out
```

32.21 IndexedDB

Browser-এর ভিতরে structured data store করার জন্য IndexedDB ব্যবহার করা যায়। এটি:

 CLIENT STORAGE CAPACITY & ARCHITECTURE

localStorage

এর চেয়ে অনেক powerful IndexedDB:

- large data store করতে পারে
- structured objects রাখতে পারে
- indexes support করে
- asynchronous
- transactions support করে

 Basic idea
JAVASCRIPT SOURCE

JAVASCRIPT

```
const request = indexedDB.open("MyDatabase", 1);
```

Database তৈরি হলে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
request.onupgradeneeded = (event) => {
  const db = event.target.result;

  db.createObjectStore("users", {
    keyPath: "id",
  });
};
```

তারপর database:

JAVASCRIPT SOURCE

JAVASCRIPT

```
request.onsuccess = (event) => {
  const db = event.target.result;

  console.log("Database opened");
};
```

♦ **Real-life**

Offline-first application:

 CLIENT STORAGE CAPACITY & ARCHITECTURE

```
Server
  ↓
IndexedDB
  ↓
Browser local database
  ↓
Offline usage
```

32.22 localStorage vs IndexedDB

Feature	localStorage	IndexedDB
Data	String	Structured data
Size	Limited	Much larger
Async	✗	✓
Transactions	✗	✓
Complex data	Limited	✓
Large applications	✗	✓

32.23 WebSockets

HTTP request-এর ক্ষেত্রে সাধারণ flow:

🌐 BROWSER API ARCHITECTURE & DATA FLOW

Client → Request
Server → Response

WebSocket persistent connection তৈরি করতে পারে:

🌐 BROWSER API ARCHITECTURE & DATA FLOW

Client ↔ Server

🌐 Basic JavaScript

JAVASCRIPT SOURCE

JAVASCRIPT

```
const socket = new WebSocket("wss://example.com/socket");
```

Connection:

JAVASCRIPT SOURCE

JAVASCRIPT

```
socket.onopen = () => {  
  console.log("Connected");  
};
```

Message:

JAVASCRIPT SOURCE

JAVASCRIPT

```
socket.onmessage = (event) => {  
  console.log(event.data);  
};
```

Send:

JAVASCRIPT SOURCE

JAVASCRIPT

```
socket.send("Hello Server");
```

Close:

JAVASCRIPT SOURCE

JAVASCRIPT

```
socket.close();
```

🌐 Real-life

Chat application:

🌐 FULL-DUPLEX WEBSOCKET PROTOCOL PIPELINE

```
User A  
  ↓  
WebSocket Server  
  ↓  
User B
```

Use cases:

- chat
- live notifications
- live dashboard
- multiplayer games
- real-time tracking

32.24 Web Share API

Mobile/browser-supported environments-এ native share dialog ব্যবহার করা যায়।

JAVASCRIPT SOURCE

JAVASCRIPT

```
async function sharePage() {
  if (navigator.share) {
    await navigator.share({
      title: "My Website",
      text: "Check this website",
      url: location.href,
    });
  }
}
```

Button:

HTML DOM MARKUP

HTML

```
<button onclick="sharePage()">Share</button>
```

32.25 Page Visibility API

User browser tab দেখছে কিনা detect করতে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
document.visibilityState;
```

Possible values:



BROWSER API ARCHITECTURE & DATA FLOW

```
visible
hidden
```

Event:

JAVASCRIPT SOURCE

JAVASCRIPT

```
document.addEventListener("visibilitychange", () => {
  if (document.hidden) {
    console.log("Tab hidden");
  } else {
    console.log("Tab visible");
  }
});
```

Real-life

Video:



BROWSER API ARCHITECTURE & DATA FLOW

```
User watches video
↓
Switches tab
↓
visibilitychange
↓
Pause video
```

Music player-এও বিভিন্ন UI optimization করা যায়।

32.26 Screen Wake Lock API

কিছু browser/device-এ screen যেন automatically sleep না করে তার জন্য Wake Lock API ব্যবহার করা যায়।

JAVASCRIPT SOURCE

JAVASCRIPT

```
let wakeLock;

async function requestWakeLock() {
  try {
    wakeLock = await navigator.wakeLock.request("screen");

    console.log("Screen wake lock active");
  } catch (error) {
    console.error(error);
  }
}
```

♦ Real-life

- navigation app
- presentation
- cooking recipe
- workout timer
- kiosk application

32.27 Navigator Object

Browser environment-এর information/capability access করতে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
navigator;
```

ব্যবহার করা হয়। উদাহরণ:

JAVASCRIPT SOURCE

JAVASCRIPT

```
console.log(navigator.language);
console.log(navigator.onLine);
console.log(navigator.userAgent);
```

আরও:

JAVASCRIPT SOURCE

JAVASCRIPT

```
navigator.geolocation;
navigator.clipboard;
navigator.mediaDevices;
navigator.permissions;
navigator.storage;
navigator.hardwareConcurrency;
```

32.28 navigator.onLine

Browser online/offline state সম্পর্কে একটি hint পাওয়া যায়:

JAVASCRIPT SOURCE

JAVASCRIPT

```
if (navigator.onLine) {  
  console.log("Online");  
} else {  
  console.log("Offline");  
}
```

Events:

JAVASCRIPT SOURCE

JAVASCRIPT

```
window.addEventListener("online", () => {  
  console.log("Back online");  
});  
  
window.addEventListener("offline", () => {  
  console.log("Offline");  
});
```

⚠ navigator.onLine internet-এর end-to-end connectivity-এর নিশ্চয়তা দেয় না।

32.29 Permissions API

Browser permission state query করার জন্য:

JAVASCRIPT SOURCE

JAVASCRIPT

```
navigator.permissions.query();
```

Example:

JAVASCRIPT SOURCE

JAVASCRIPT

```
navigator.permissions  
  .query({  
    name: "geolocation",  
  })  
  .then((result) => {  
    console.log(result.state);  
  });
```

Possible:

🌐 BROWSER API ARCHITECTURE & DATA FLOW

```
granted  
denied  
prompt
```

32.30 Browser API Security

Advanced Browser API ব্যবহার করার সময় security গুরুত্বপূর্ণ।

♦ Example:

Geolocation:

🌐 GEOLOCATION HARDWARE INTEGRATION FLOW

User permission
↓
Location access

Clipboard:

🌐 BROWSER API ARCHITECTURE & DATA FLOW

User interaction / browser security
↓
Clipboard access

Notification:

🌐 BROWSER API ARCHITECTURE & DATA FLOW

Permission
↓
Notification

তাই browser API মানেই unrestricted access নয়।

32.31 Practical Project — Image Preview

HTML:

HTML DOM MARKUP

HTML

```
<input type="file" id="imageInput" />

<img id="preview" width="300" alt="Preview" />
```

JavaScript:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const input = document.querySelector("#imageInput");

const preview = document.querySelector("#preview");

input.addEventListener("change", () => {
  const file = input.files[0];

  if (!file) return;

  const url = URL.createObjectURL(file);

  preview.src = url;

  preview.onload = () => {
    URL.revokeObjectURL(url);
  };
});
```

♦ এখানে কী শিখলাম?

🌐 FILE API & BLOB PROCESSING PIPELINE

```
<input type=file>
  ↓
File
  ↓
Object URL
  ↓
Image Preview
  ↓
revokeObjectURL()
```

32.32 Practical Project — Copy Button

HTML:

HTML DOM MARKUP

HTML

```
<p id="text">JavaScript is powerful.</p>

<button id="copyBtn">Copy</button>
```

JavaScript:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const text = document.querySelector("#text");

const button = document.querySelector("#copyBtn");

button.addEventListener("click", async () => {
  try {
    await navigator.clipboard.writeText(text.textContent);

    button.textContent = "Copied!";

    setTimeout(() => {
      button.textContent = "Copy";
    }, 1500);
  } catch (error) {
    console.error(error);
  }
});
```

32.33 Practical Project — Lazy Loading

HTML DOM MARKUP

HTML

```

```

JAVASCRIPT SOURCE

JAVASCRIPT

```
const images = document.querySelectorAll(".lazy");

const observer = new IntersectionObserver((entries) => {
  entries.forEach((entry) => {
    if (!entry.isIntersecting) return;

    const img = entry.target;

    img.src = img.dataset.src;

    observer.unobserve(img);
  });
});

images.forEach((img) => {
  observer.observe(img);
});
```

এটি large website-এর performance improve করতে সাহায্য করতে পারে।

32.34 Practical Project — Tab Communication

◆ Tab 1

JAVASCRIPT SOURCE

JAVASCRIPT

```
const channel = new BroadcastChannel("notifications");

channel.postMessage({
  message: "New task added",
});
```

◆ Tab 2

JAVASCRIPT SOURCE

JAVASCRIPT

```
const channel = new BroadcastChannel("notifications");

channel.onmessage = (event) => {
  console.log(event.data.message);
};
```

Output:

🌐 BROWSER API ARCHITECTURE & DATA FLOW

New task added

32.35 Browser APIs — কোনটা কখন?

Problem	API
Location	Geolocation
URL parse	URL
Query parameters	URLSearchParams
Browser route	History API
Copy text	Clipboard
Notification	Notifications
Viewport visibility	IntersectionObserver
DOM changes	MutationObserver
Element resize	ResizeObserver
File upload	File API
Read file	FileReader
Binary data	Blob
Temporary file URL	Object URL
Drag file	Drag & Drop
Heavy computation	Web Worker
Tab communication	BroadcastChannel
Large local database	IndexedDB
Real-time communication	WebSocket
Native sharing	Web Share
Tab visibility	Page Visibility
Prevent screen sleep	Wake Lock
Browser capabilities	Navigator
Permission state	Permissions API

🔥 32.36 MUST KNOW এই Chapter থেকে এগুলো অবশ্যই ভালোভাবে জানতে হবে:

♦ ★ 1. URL

JAVASCRIPT SOURCE

JAVASCRIPT

```
new URL();
```

♦ ★ 2. URLSearchParams

JAVASCRIPT SOURCE

JAVASCRIPT

```
new URLSearchParams();
```

♦ ★ 3. History API

JAVASCRIPT SOURCE

JAVASCRIPT

```
history.pushState();
history.replaceState();
```

♦ ★ 4. IntersectionObserver

Lazy loading এবং viewport detection-এর জন্য।

♦ ★ 5. MutationObserver

DOM changes detect করার জন্য।

♦ ★ 6. ResizeObserver

Element size change detect করার জন্য।

♦ ★ 7. File API

```
input.files;
```

◆ ★ 8. FileReader

File content পড়তে।

◆ ★ 9. Blob

Binary-like data handle করতে।

◆ ★ 10. Web Worker

Heavy computation আলাদা worker context-এ চালাতে।

◆ ★ 11. IndexedDB

Browser-side structured database।

◆ ★ 12. WebSocket

Real-time two-way communication।

◆ ★ 13. Clipboard API

Copy/paste।

◆ ★ 14. BroadcastChannel

Multiple tabs communication। # 🍷 Quick Cheat Sheet

```
// Location
navigator.geolocation.getCurrentPosition( ... );

// URL
const url = new URL("https://example.com");

// Query params
const params = new URLSearchParams("?id=10");

// History
history.pushState({}, "", "/home");

// Clipboard
await navigator.clipboard.writeText("Hello");

// Notification
await Notification.requestPermission();

// Intersection
new IntersectionObserver(callback);

// DOM mutation
new MutationObserver(callback);

// Resize
new ResizeObserver(callback);

// File
input.files[0];

// FileReader
new FileReader();

// Blob
new Blob(["Hello"]);

// Object URL
URL.createObjectURL(file);

// Worker
new Worker("worker.js");

// Broadcast
new BroadcastChannel("channel");

// IndexedDB
indexedDB.open("Database");

// WebSocket
new WebSocket("wss://example.com");

// Share
navigator.share({ ... });

// Visibility
document.visibilityState;

// Wake Lock
navigator.wakeLock.request("screen");

// Permissions
navigator.permissions.query({ ... });
```

📁 Practice Set

◆ Beginner

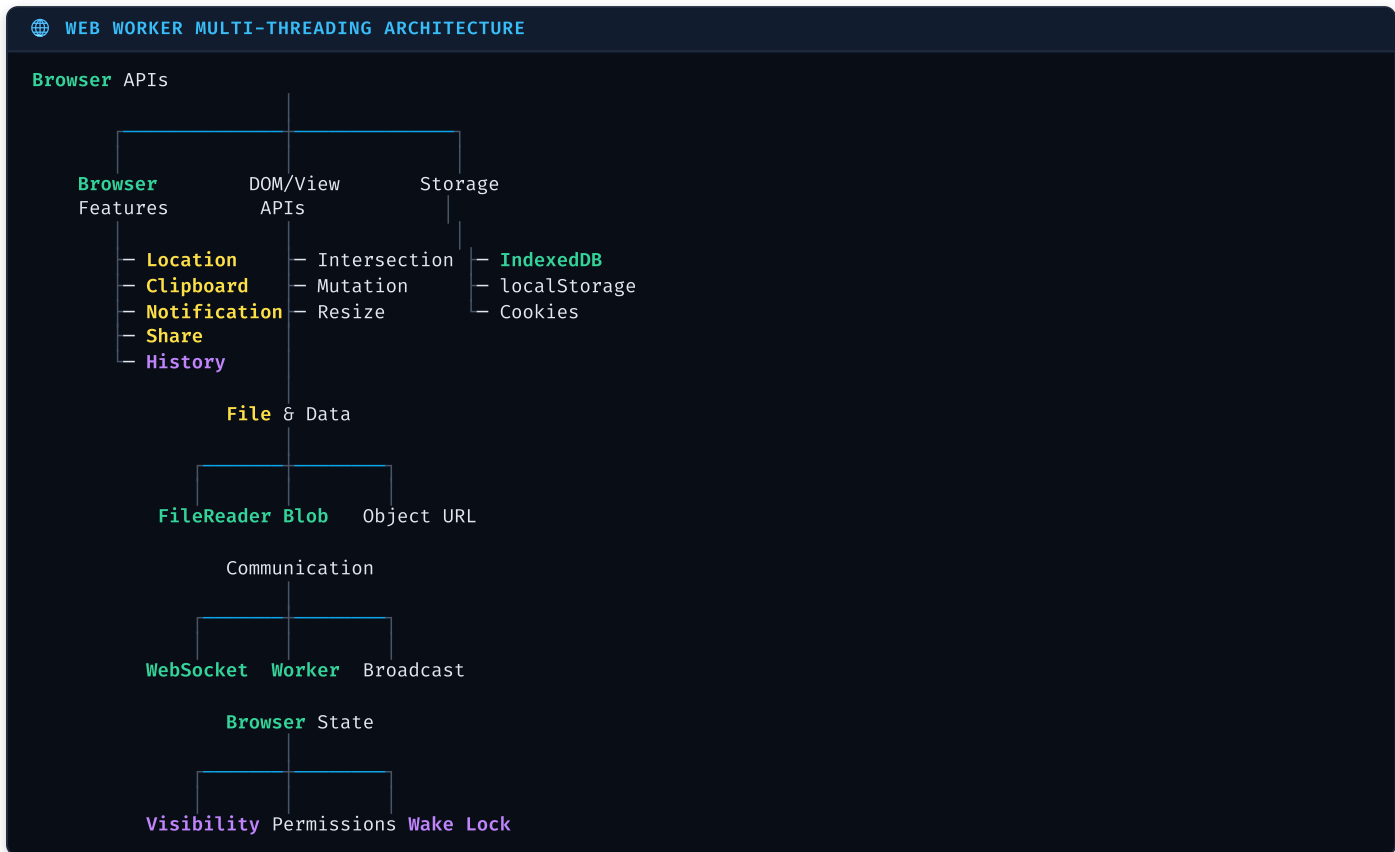
1. Current browser URL থেকে hostname বের করুন। 2. URL থেকে id query parameter বের করুন। 3. একটি Copy button তৈরি করুন। 4. একটি image upload করে preview দেখান। 5. একটি notification button তৈরি করুন।

◆ Intermediate

6. IntersectionObserver দিয়ে lazy image loader তৈরি করুন। 7. MutationObserver দিয়ে DOM change detect করুন। 8. ResizeObserver দিয়ে box-এর width display করুন। 9. Drag & Drop image uploader তৈরি করুন। 10. BroadcastChannel ব্যবহার করে দুই tab-এর মধ্যে message পাঠান।

◆ Advanced

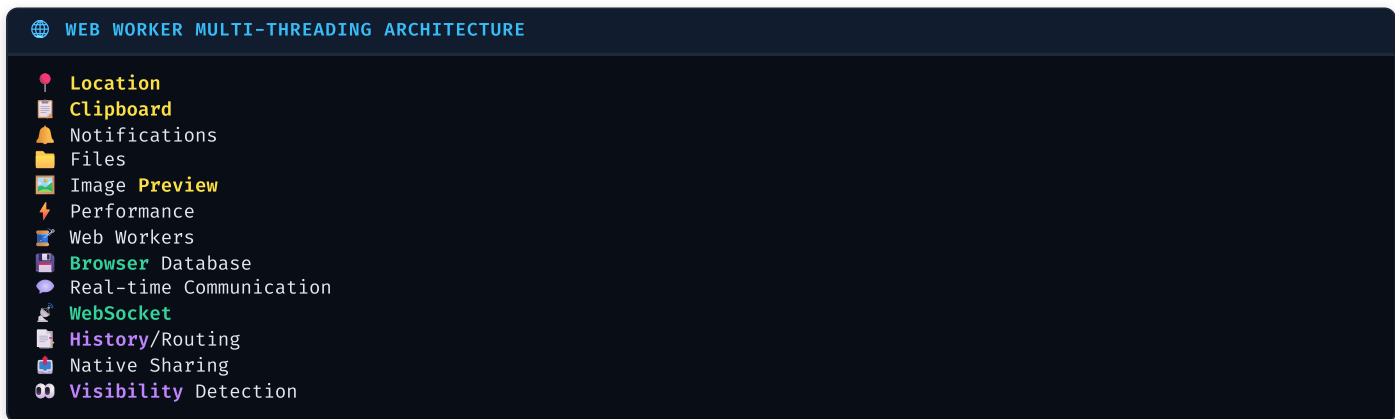
11. Web Worker ব্যবহার করে large calculation করুন। 12. IndexedDB-তে user data save করুন। 13. WebSocket দিয়ে real-time chat UI তৈরি করুন। 14. History API দিয়ে simple SPA router তৈরি করুন। 15. File API + Blob + Object URL ব্যবহার করে browser-based file preview/download system তৈরি করুন। # 🧠 Final Mental Map পুরো Chapter-টা এভাবে মনে রাখুন:



🌐 🧠 Chapter 32-এর মূল কথা

💡 Browser API = JavaScript + Browser-এর extra capabilities

JavaScript দিয়ে শুধু calculation বা DOM manipulation করাই নয়—Browser APIs ব্যবহার করে তুমি:



এর মতো real-world functionality তৈরি করতে পারবে। পরবর্তী Chapter 33 → JavaScript Web Performance & Optimization — এখানে আমরা দেখব কীভাবে JavaScript application দ্রুত, lightweight, responsive এবং production-ready করা যায়।

32.36 🔥 MUST KNOW — ব্রাউজার প্ল্যাটফর্মের ১৪টি প্রধান শক্ত

মডার্ন ওয়েব অ্যাপ্লিকেশনের জন্য যেসব ব্রাউজার এপিআই প্রতিটি ইঞ্জিনিয়ারের নখদর্পণে থাকা প্রয়োজন:

📍 1. Navigation & Hardware

- **URL & URLSearchParams:** Query string parsing
- **History API:** pushState/popstate SPA router
- **Geolocation:** GPS coordinate tracking
- **Clipboard API:** Async copy/paste text
- **Notifications:** Desktop push notifications

🔗 2. Observers & File Pipeline

- **IntersectionObserver:** Lazy loading & infinite scroll
- **MutationObserver:** DOM tree mutation listening
- **ResizeObserver:** Container query size changes
- **File API & FileReader:** Local file stream read
- **Blob & Object URL:** In-memory binary URLs

🔗 3. Concurrency & Storage

- **Web Workers:** Heavy tasks on separate threads
- **BroadcastChannel:** Multi-tab real-time sync
- **IndexedDB:** High-capacity client NoSQL DB
- **WebSockets:** Full-duplex instant communication
- **Page Visibility:** Background tab power saving

32.Cheat Quick Cheat Sheet — Browser APIs Rapid Reference

দৈনন্দিন প্রোডাকশন ডেভেলপমেন্টে বহুল ব্যবহৃত ব্রাউজার এপিআই মেথডগুলোর দ্রুত সিনট্যাক্স গাইড:

BROWSER APIS RAPID CHEAT SHEET

JAVASCRIPT

```
// Location
navigator.geolocation.getCurrentPosition( ... );

// URL
const url = new URL("https://example.com");

// Query params
const params = new URLSearchParams("?id=10");

// History
history.pushState({}, "", "/home");

// Clipboard
await navigator.clipboard.writeText("Hello");

// Notification
await Notification.requestPermission();

// Intersection
new IntersectionObserver(callback);

// DOM mutation
new MutationObserver(callback);

// Resize
new ResizeObserver(callback);

// File
input.files[0];

// FileReader
new FileReader();

// Blob
new Blob(["Hello"]);

// Object URL
URL.createObjectURL(file);

// Worker
new Worker("worker.js");

// Broadcast
new BroadcastChannel("channel");

// IndexedDB
indexedDB.open("Database");

// WebSocket
new WebSocket("wss://example.com");

// Share
navigator.share({ ... });

// Visibility
document.visibilityState;

// Wake Lock
navigator.wakeLock.request("screen");

// Permissions
navigator.permissions.query({ ... });
```

রিয়েল-ওয়ার্ল্ড ব্রাউজার এপিআই দক্ষতায় পূর্ণাঙ্গ পারদর্শিতা অর্জন করতে ১৫টি ইন্টারভিউ প্রবলেমের সমাধান:

Practice 1: URL Hostname Extraction

BEGINNER

? বর্তমান ব্রাউজার URL থেকে hostname বের করার নিরাপদ উপায় কী?

PRACTICE 1 IMPLEMENTATION

JAVASCRIPT

```
// Method 1: Using window.location directly
const host1 = window.location.hostname; // e.g. "developer.mozilla.org"

// Method 2: Using the modern URL API for any string
const parsedUrl = new URL("https://github.com/Shariar-Ahamed/Note-Book?tab=readme");
console.log(parsedUrl.hostname); // "github.com"
console.log(parsedUrl.pathname); // "/Shariar-Ahamed/Note-Book"
```

▶ CONSOLE / EVENT OUTPUT

OUTPUT

```
github.com
```

🔍 Model Architecture & Solution Breakdown:

১. 'window.location.hostname' প্রোপার্টি দিয়ে বর্তমান পেজের ডোমেইন বা আইপি পাওয়া যায়।
২. যেকোনো ভয়নামিক URL স্ট্রিং পার্স করার জন্য 'new URL(urlString)' সবচেয়ে নিরাপদ ও আদর্শ মেকানিজম।

Practice 2: URL Query Parameter Parsing with URLSearchParams

BEGINNER

? URL থেকে 'id' কুয়েরি প্যারামিটার কীভাবে সহজে বের করবে?

PRACTICE 2 IMPLEMENTATION

JAVASCRIPT

```
const searchParams = new URLSearchParams(window.location.search);
// Example URL: https://site.com/products?id=42&category=laptops

if (searchParams.has("id")) {
  const productId = searchParams.get("id");
  console.log("Found Product ID:", productId); // "42"
}
```

▶ CONSOLE / EVENT OUTPUT

OUTPUT

```
Found Product ID: 42
```

🔍 Model Architecture & Solution Breakdown:

১. 'new URLSearchParams(window.location.search)' দিয়ে ব্রাউজারের সার্চ স্ট্রিং তাৎক্ষণিকভাবে ম্যাপে কনভার্ট হয়।
২. 'get("id")' দিয়ে সরাসরি ভ্যালু পাওয়া যায় এবং 'has("id")' দিয়ে উপস্থিতি চেক করা যায়।

Practice 3: Async Clipboard API One-Click Copy Button

BEGINNER

? আধুনিক ব্রাউজারে একটি সুরক্ষিত ও অ্যাসিঙ্ক কপি বাটন কীভাবে তৈরি করবে?

PRACTICE 3 IMPLEMENTATION

JAVASCRIPT

```
async function copyToClipboard(textToCopy) {
  try {
    await navigator.clipboard.writeText(textToCopy);
    console.log("Copied to clipboard successfully!");
  } catch (err) {
    console.error("Clipboard permission denied or error:", err);
  }
}

// User trigger:
document.querySelector("#copyBtn").addEventListener("click", () => {
  copyToClipboard("npm install antigravity-ide");
});
```

▶ CONSOLE / EVENT OUTPUT

OUTPUT

```
Copied to clipboard successfully!
```

🔍 Model Architecture & Solution Breakdown:

১. 'navigator.clipboard.writeText()' একটি প্রমিজ রিটার্ন করে এবং ব্যাকগ্রাউন্ডে নন-ব্লকিংভাবে কাজ করে।
২. সিকিউরিটি নোট: এটি কেবল HTTPS এবং ইউজার ইন্টারঅ্যাকশন (click gesture)-এর ভেতরেই কাজ করতে পারে।

Practice 4: Local Image File Preview with FileReader

BEGINNER

? ইউজার ইমেজ ফাইল আপলোড করলে তা সার্ভারে না পাঠিয়ে লোকাল প্রিভিউ কীভাবে দেখাবে?

PRACTICE 4 IMPLEMENTATION

JAVASCRIPT

```
const fileInput = document.querySelector("#imageInput");
const previewImg = document.querySelector("#preview");

fileInput.addEventListener("change", (e) => {
  const file = e.target.files[0];
  if (file && file.type.startsWith("image/")) {
    const reader = new FileReader();
    reader.onload = (event) => {
      previewImg.src = event.target.result; // Base64 DataURL
    };
    reader.readAsDataURL(file);
  }
});
```

► CONSOLE / EVENT OUTPUT

OUTPUT

```
data:image/png;base64,iVBORw0KGgo...
```

🔍 Model Architecture & Solution Breakdown:

১. 'e.target.files[0]' দিয়ে File অবজেক্ট পাওয়া যায়।
২. 'FileReader'-এর 'readAsDataURL()' মেথড ইমেজটিকে Base64 ফ্রিংয়ে রূপান্তর করে ইমেজ ট্যাগের src-তে বসিয়ে দেয়।

Practice 5: Web Notifications API Permission & Trigger

BEGINNER

? ইউজারের অনুমতি নিয়ে কীভাবে ব্রাউজার নোটিফিকেশন পাঠাবে?

PRACTICE 5 IMPLEMENTATION

JAVASCRIPT

```
async function triggerDesktopNotification(title, message) {
  if (!("Notification" in window)) return;

  let perm = Notification.permission;
  if (perm === "default") {
    perm = await Notification.requestPermission();
  }

  if (perm === "granted") {
    new Notification(title, {
      body: message,
      icon: "/assets/bell.png"
    });
  }
}
```

► CONSOLE / EVENT OUTPUT

OUTPUT

```
Desktop Push Notification Displayed
```

🔍 Model Architecture & Solution Breakdown:

১. 'Notification.requestPermission()' ইউজারের কাছে প্রপন্স্ট দেখায় ('granted', 'denied', বা 'default')।
২. অনুমতি থাকলে 'new Notification(title, options)' কল করে সিস্টেম নোটিফিকেশন প্রদর্শন করা যায়।

Practice 6: High-Performance Lazy Image Loader with IntersectionObserver

INTERMEDIATE

? স্ক্রল ইভেন্ট লিসেনার ছাড়া শুধুমাত্র IntersectionObserver দিয়ে ইমেজ লেজি লোড কীভাবে করবে?

PRACTICE 6 IMPLEMENTATION

JAVASCRIPT

```
const observer = new IntersectionObserver((entries, obs) => {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      const img = entry.target;
      img.src = img.dataset.src; // Swap placeholder with real image
      img.classList.add("loaded");
      obs.unobserve(img); // Disconnect after load to save resources
    }
  });
}, { threshold: 0.1 });

document.querySelectorAll("img.lazy").forEach(img => observer.observe(img));
```

► CONSOLE / EVENT OUTPUT

OUTPUT

Zero-lag Lazy Loading Active (Images load as they enter viewport)

🔍 Model Architecture & Solution Breakdown:

- IntersectionObserver ব্রাউজারের ইন্টারনাল রেন্ডারিং থ্রেডে চলে, ফলে স্ক্রল ইভেন্টের মতো মেইন থ্রেড ব্লক করে না।
- ইমেজ ভিউপোর্টে আসামাত্র 'entry.isIntersecting' সত্য হয়; ডেটাসেট থেকে আসল URL বসিয়ে 'unobserve()' কল করে ক্লিনআপ করা হয়।

Practice 7: DOM Tree Mutation Tracking with MutationObserver

INTERMEDIATE

? কোনো এলিমেন্টে নতুন চাইল্ড নোড যুক্ত বা মুছে ফেলা হলে তা কীভাবে ট্র্যাক করবে?

PRACTICE 7 IMPLEMENTATION

JAVASCRIPT

```
const targetContainer = document.querySelector("#chatBox");

const observer = new MutationObserver((mutations) => {
  mutations.forEach(mutation => {
    if (mutation.type === "childList") {
      console.log("Nodes added:", mutation.addedNodes.length);
      console.log("Nodes removed:", mutation.removedNodes.length);
    }
  });
});

observer.observe(targetContainer, { childList: true, subtree: true });
```

► CONSOLE / EVENT OUTPUT

OUTPUT

Nodes added: 1 | Chat Auto-Scrolled

🔍 Model Architecture & Solution Breakdown:

- MutationObserver পুরানো ডিপ্রিকিটেড Mutation Events-এর তুলনায় হাজার গুণ দ্রুত ও পারফরম্যান্ট।
- 'childList: true' এবং 'subtree: true' অপশন দিয়ে যেকোনো নেস্টেড চাইল্ড পরিবর্তনের ব্যাচ রেসপন্স পাওয়া যায়।

Practice 8: Responsive Container Sizing with ResizeObserver

INTERMEDIATE

? উইন্ডো রিসাইজ ছাড়াই নির্দিষ্ট কোনো কার্ড বা ডিভাইসের সাইজ পরিবর্তন কীভাবে মনিটর করবে?

PRACTICE 8 IMPLEMENTATION

JAVASCRIPT

```
const card = document.querySelector(".resizable-card");

const resizeObserver = new ResizeObserver((entries) => {
  for (let entry of entries) {
    const { width, height } = entry.contentRect;
    card.querySelector(".size-label").textContent = `${Math.round(width)}px × ${Math.round(height)}px`;
  }
});

resizeObserver.observe(card);
```

► CONSOLE / EVENT OUTPUT

OUTPUT

Real-time dimensions: 320px × 180px

🔍 Model Architecture & Solution Breakdown:

1. ResizeObserver সরাসরি এলিমেন্টের 'contentRect' প্রদান করে।
2. এটি আধুনিক Container Queries এবং রেসপনসিভ কম্পোনেন্ট ডিজাইনের জন্য সর্বোত্তম।

Practice 9: Drag and Drop File Dropzone Uploader

INTERMEDIATE

? ব্রাউজারে একটি ড্র্যাগ অ্যান্ড ড্রপ ফাইল ড্রপজোন কীভাবে তৈরি করবে?

PRACTICE 9 IMPLEMENTATION

JAVASCRIPT

```
const dropzone = document.querySelector("#dropzone");

dropzone.addEventListener("dragover", (e) => {
  e.preventDefault(); // Required to allow drop!
  dropzone.classList.add("drag-active");
});

dropzone.addEventListener("dragleave", () => {
  dropzone.classList.remove("drag-active");
});

dropzone.addEventListener("drop", (e) => {
  e.preventDefault();
  dropzone.classList.remove("drag-active");

  const files = e.dataTransfer.files;
  console.log("Dropped files count:", files.length);
  handleFileUpload(files[0]);
});
```

► CONSOLE / EVENT OUTPUT

OUTPUT

Dropped files count: 1 | Upload triggered

🔍 Model Architecture & Solution Breakdown:

1. 'dragover' ইভেন্টে 'e.preventDefault()' কল করা বাধ্যতামূলক, নইলে ব্রাউজার ডিফল্টভাবে ফাইলটি ওপেন করে ফেলে।
2. 'e.dataTransfer.files' দিয়ে সরাসরি ড্রপ করা ফাইলগুলোর লিস্ট অ্যাক্সেস করা যায়।

Practice 10: Multi-Tab State Synchronization with BroadcastChannel

INTERMEDIATE

? একই অ্যাপ্লিকেশনের দুটি ওপেন ট্যাবের মধ্যে কীভাবে লাইভ মেসেজ পাস করবে?

PRACTICE 10 IMPLEMENTATION

JAVASCRIPT

```
// Run this code in Tab 1 and Tab 2:
const authChannel = new BroadcastChannel("auth_sync_channel");

// Tab 2 (Sender): When user logs out
function logoutUser() {
  authChannel.postMessage({ type: "LOGOUT", timestamp: Date.now() });
}

// Tab 1 (Listener): Auto logout
authChannel.onmessage = (event) => {
  if (event.data.type === "LOGOUT") {
    console.log("User logged out in another tab! Redirecting to login...");
    window.location.reload();
  }
};
```

▶ CONSOLE / EVENT OUTPUT

OUTPUT

User logged out in another tab! Redirecting to login...

🔍 Model Architecture & Solution Breakdown:

- ১. BroadcastChannel একই অরিজিনের সব উইন্ডো, ট্যাব এবং আইফ্রেমের মধ্যে নন-ব্লকিং ব্রডকাস্ট মেসেজিং নিশ্চিত করে।
- ২. সেশন সিল্ক, থিম চেঞ্জ বা শপিং কার্ট আপডেটের জন্য এটি localStorage ইভেন্টের চেয়ে শতভাগ দ্রুত।

Practice 11: Heavy Computation Offloading with Web Workers

ADVANCED 🔥

? ভারী কোনো হিসাব মেইন থ্রেড ব্লক না করে কীভাবে Web Worker দিয়ে চালাবে?

PRACTICE 11 IMPLEMENTATION

JAVASCRIPT

```
// main.js:
const worker = new Worker("heavy_worker.js");

worker.postMessage({ number: 45 }); // Offload heavy task
console.log("UI remains 100% responsive and silky smooth!");

worker.onmessage = (e) => {
  console.log("Calculation Result from Worker:", e.data.result);
};

// heavy_worker.js (Isolated background thread):
self.onmessage = (e) => {
  function fibonacci(n) {
    return n <= 1 ? n : fibonacci(n - 1) + fibonacci(n - 2);
  }

  const result = fibonacci(e.data.number);
  self.postMessage({ result });
};
```

▶ CONSOLE / EVENT OUTPUT

OUTPUT

UI remains 100% responsive! | Result: 1134903170

🔍 Model Architecture & Solution Breakdown:

- ১. Web Worker আলাদা OS থ্রেড চলে, ফলে ভারী ম্যাথমেটিক্যাল ক্যালকুলেশন বা ইমেজ প্রসেসিং মেইন UI থ্রেডকে (60fps) কখনোই ফ্রিজ করে না।
- ২. এটি মেসেজ পাসিং ('postMessage') দিয়ে মেইন থ্রেডের সাথে যোগাযোগ করে।

Practice 12: Structured Storage with IndexedDB

ADVANCED 🔥

? ব্রাউজারে বড় আকারের অবজেক্ট কীভাবে IndexedDB-তে স্টোর ও রিট্রিভ করবে?

PRACTICE 12 IMPLEMENTATION

JAVASCRIPT

```
const req = indexedDB.open("OfflineStore", 1);

req.onupgradeneeded = (e) => {
  const db = e.target.result;
  db.createObjectStore("notes", { keyPath: "id", autoIncrement: true });
};

req.onsuccess = (e) => {
  const db = e.target.result;
  const tx = db.transaction("notes", "readwrite");
  const store = tx.objectStore("notes");

  store.add({ title: "Master JS Notes", date: new Date().toISOString() });

  tx.oncomplete = () => console.log("Transaction saved to IndexedDB!");
};
```

► CONSOLE / EVENT OUTPUT

OUTPUT

Transaction saved to IndexedDB!

🔍 Model Architecture & Solution Breakdown:

- IndexedDB একটি সম্পূর্ণ ট্রানজ্যাকশনাল NoSQL অবজেক্ট ডেটাবেস যা গিগাবাইট সাইজের ডেটা অফলাইনে ধরে রাখতে পারে।
- এটি অ্যাসিঙ্ক্রোনাস হওয়ায় ব্রাউজার পারফরম্যান্স অক্ষুণ্ণ রাখে।

Practice 13: Real-Time Full-Duplex Chat Client with WebSockets

ADVANCED 🔥

? সার্ভারের সাথে ফুল-ডুপ্লেক্স WebSocket কানেকশন কীভাবে হ্যান্ডল করবে?

PRACTICE 13 IMPLEMENTATION

JAVASCRIPT

```
const socket = new WebSocket("wss://echo.websocket.events");

socket.onopen = () => {
  console.log("Connected to Real-time WebSocket Server!");
  socket.send(JSON.stringify({ text: "Hello from Client!" }));
};

socket.onmessage = (event) => {
  const msg = JSON.parse(event.data);
  console.log("Message received from Server:", msg.text);
};

socket.onerror = (err) => console.error("Socket error:", err);
socket.onclose = () => console.log("Socket connection terminated cleanly.");
```

► CONSOLE / EVENT OUTPUT

OUTPUT

Connected to WebSocket! | Message received: Hello from Client!

🔍 Model Architecture & Solution Breakdown:

- WebSocket একটি স্থায়ী TCP কানেকশন তৈরি করে যাতে সার্ভার ও ক্লায়েন্ট উভয়ই যেকোনো সময় ডেটা পুশ করতে পারে।
- এটি সাধারণ HTTP লং-পোলিংয়ের বিশাল ওভারহেড দূর করে সাব-মিলিমিটারে ডেটা ট্রান্সমিট করে।

Practice 14: Client-Side SPA Routing Engine with History API

ADVANCED 🔥

? পেজ রিলোড ছাড়া History API দিয়ে সিঙ্গেল পেজ রাউটিং কীভাবে কাজ করে?

PRACTICE 14 IMPLEMENTATION

JAVASCRIPT

```
function navigateTo(urlPath) {
  history.pushState({ path: urlPath }, "", urlPath);
  renderPage(urlPath);
}

window.addEventListener("popstate", (event) => {
  // Triggered when user presses Browser Back or Forward buttons!
  renderPage(window.location.pathname);
});

function renderPage(path) {
  console.log("Rendered SPA view for route:", path);
}
```

► CONSOLE / EVENT OUTPUT

OUTPUT

Rendered SPA view for route: /dashboard

🔍 Model Architecture & Solution Breakdown:

১. 'history.pushState()' রাউজারের অ্যাক্সেস বার ও যিষ্টি ট্র্যাকে নতুন URL যোগ করে কোনো সার্ভার রিলোড ছাড়াই।
২. 'popstate' ইভেন্ট হ্যান্ডলার রাউজারের Back/Forward বাটনের ন্যাভিগেশন নিখুঁতভাবে হ্যান্ডল করে।

Practice 15: Dynamic In-Memory File Generation & Download with Blob & Object URL

ADVANCED 🔥

? সার্ভার ছাড়া ব্রাউজার মেমরিতে টেক্সট/JSON ফাইল বানিয়ে স্বয়ংক্রিয়ভাবে কীভাবে ডাউনলোড করাবে?

PRACTICE 15 IMPLEMENTATION

JAVASCRIPT

```
function downloadJsonFile(dataObj, filename = "export.json") {
  const jsonStr = JSON.stringify(dataObj, null, 2);
  const blob = new Blob([jsonStr], { type: "application/json" });

  const downloadUrl = URL.createObjectURL(blob);
  const tempLink = document.createElement("a");
  tempLink.href = downloadUrl;
  tempLink.download = filename;

  document.body.appendChild(tempLink);
  tempLink.click();

  // Clean up memory
  document.body.removeChild(tempLink);
  URL.revokeObjectURL(downloadUrl);
  console.log("File downloaded & memory revoked cleanly.");
}
```

► CONSOLE / EVENT OUTPUT

OUTPUT

File downloaded & memory revoked cleanly.

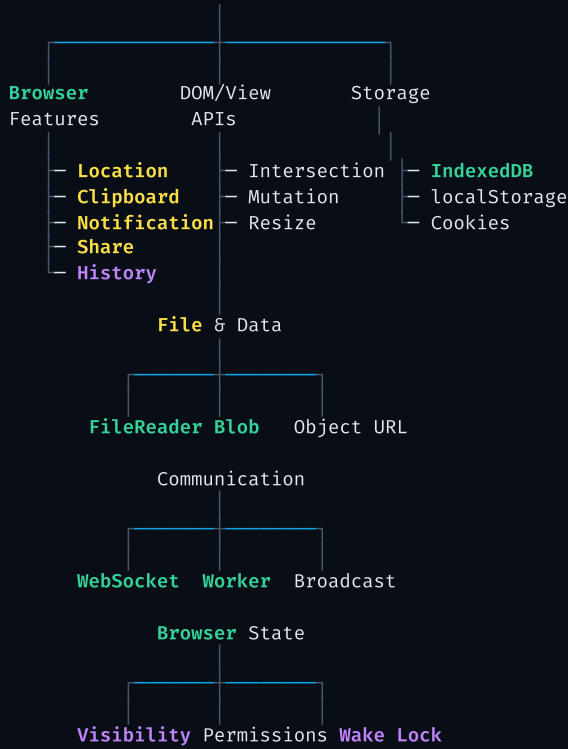
🔍 Model Architecture & Solution Breakdown:

১. 'new Blob()' ক্লায়েন্ট-সাইড মেমরিতে বাইনারি ডেটা কন্টেইনার তৈরি করে।
২. 'URL.createObjectURL(blob)' একটি অস্থায়ী 'blob:http...' URL তৈরি করে যা লিঙ্কের ডাউনলোড হিসেবে কাজ করে।
৩. মেমরি লিক এড়াতে ডাউনলোড শেষে 'URL.revokeObjectURL()' কল করা প্রোডাকশন কোডের গোল্ডেন স্ট্যান্ডার্ড।

ব্রাউজার ফিচার, ভিউপোর্ট অবজারভার, ডেটা স্টোরেজ ও থ্রেডিং কমিউনিকেশনের সামগ্রিক মানচিত্র:

🌐 BROWSER APIS GLOBAL ARCHITECTURAL COORDINATION TREE

Browser APIs



🚀 Chapter 32-এর মূল দর্শন:

Browser API = JavaScript + Browser-এর extra capabilities! JavaScript কেবল সাধারণ ক্যালকুলেশন বা DOM ম্যানিপুলেশন নয়—লোকেশন ট্র্যাকিং, পুশ নোটিফিকেশন, ব্যাকগ্রাউন্ড মাল্টি-থ্রেডিং, জিগাবাইট স্টোরেজ ও রিয়েল-টাইম ফুল-ডুপ্লেক্স কমিউনিকেশনের মাধ্যমে পূর্ণাঙ্গ নেটিভ অ্যাপ সমতুল্য অভিজ্ঞতা দেয়।

🚀 Next Chapter Preview:

Chapter 33 — JavaScript Web Performance & Optimization

পরবর্তী চ্যাপ্টারে আমরা শিখব Critical Rendering Path, Repaint & Reflow, Debounce/Throttle, Code Splitting, Bundle Analysis, Memory Profiling এবং Web Vitals (LCP, FID/INP, CLS)।